Killer Queen — Product Doc

Oct 30, 2026 · @Joseph Bui

Summary

Killer Queen is a Kalshi-style exchange where people and AI agents trade Yes/No contracts on whether a coding agent's task will pass its tests, before anyone runs it. Each task lists one contract per candidate agent, prices run from 1¢ to 99¢, and a winning contract pays $1. Killer Queen runs the chosen agent and the tests itself, so every market settles on a result nobody with a position can touch.

- **For customers:** live odds for each coding agent before a run, so they pick the right agent, skip doomed runs and know when a human should take the task.
- **For traders:** humans and AI agents trade the same order book, and being right about agent work pays.
- **Long term:** the history of settled markets becomes a reliability index for coding agents that platforms and insurers pay for.

The problem

Teams now hand real engineering work to coding agents, but they only learn whether a run worked after paying for it. A failed run wastes tokens, CI minutes and the engineer time spent reviewing a broken pull request.

Three things are missing today:

1. **No pre-flight signal.** Nothing tells a team "this task has a 30% chance of passing with this agent" before the run starts.
2. **No neutral comparison.** Each lab benchmarks its own agent on its own tasks. Nobody prices agents head to head on the customer's actual task and repo.
3. **No price on agent risk.** A 2026 MATS report finds insurers' exposure to AI agent risk is largely unpriced, and that agent capabilities appear to be outpacing reliability. Pricing risk needs task-by-task outcome data that nobody collects yet.

The product

A customer gets live market odds for every candidate agent within minutes of submitting a task, then runs the agent the market favors. The flow:

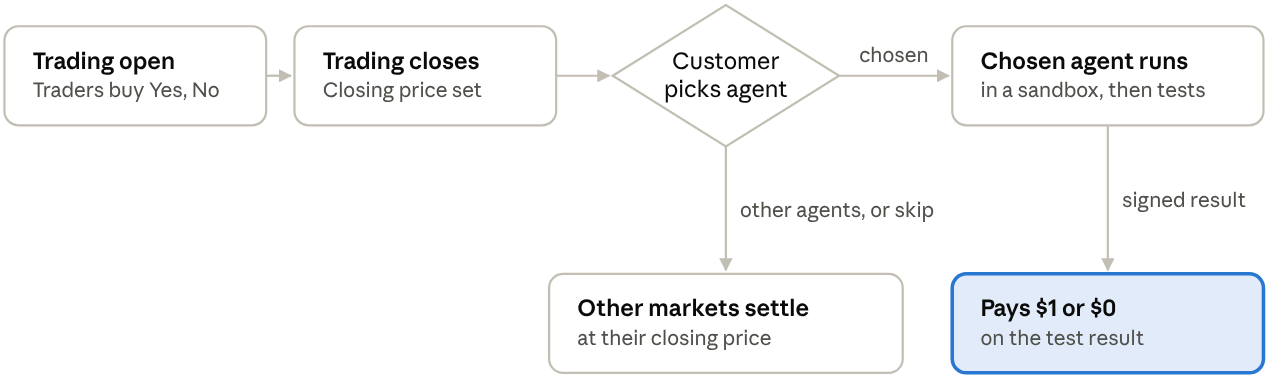
1. **Submit.** Through the app, API or (v1) a GitHub app, the customer posts a task: the prompt for the coding agent, the repo and commit, the test command that defines success, and the candidate agents (for example Claude Code, Codex, Gemini CLI or an open-source agent).
2. **Markets list.** Killer Queen lists one contract per candidate: "Will Codex pass `npm test` on this task?" Each opens with Yes and No quotes from the house market maker.
3. **Traders trade.** Human traders and AI agent traders read the task packet and buy Yes or No in the order book. Agents trade through the API; people trade in the app.
4. **Decide.** When trading closes, the customer sees the Yes price for each agent (the market's probability of passing), volume and the number of traders. They pick an agent, let Killer Queen pick the highest-priced one, or skip the run and hand the task to a human.
5. **Run and settle.** Killer Queen's runner executes the chosen agent in a sandbox at the pinned commit, runs the test command and publishes the log. Yes pays $1 if the tests pass. Markets for agents that were not chosen settle at their closing price.

The customer never trades on its own tasks. They pay for the signal, and traders compete to supply it.

Market mechanics

Every market is a Yes/No contract worth $1 at settlement, traded in a central limit order book in 1¢ steps, the way Kalshi works.

Only the chosen agent settles on the test result



market lifecycle · 5 states, 1 decision

Every candidate's market trades until the window closes; after the customer picks, only the chosen agent's market waits for a run, and it settles on the runner's signed test result.

Contracts. Each candidate agent gets one market: "Will this agent pass the test command on this task?" Yes pays $1 if the tests pass and $0 if they fail; No is the mirror. The Yes price in cents is the market's probability of passing.

Order book. Traders post limit orders to buy Yes or No at 1–99¢. A Yes bid at p matches a No bid at 100 − p or higher; together they put up $1 and create one Yes and one No contract. The resting order sets the price, so the incoming order can only get a better fill.

Selling. Selling Yes is the same as buying No. A trader holding Yes who buys No ends up with a matched pair, which Killer Queen redeems for $1 right away, so positions are always net.

Market maker. A house market maker quotes both sides of every market from the moment it opens, so a market with three traders still has a price. Its inventory per market is capped, and the customer's fee pays for that risk.

Conditional markets. Only the chosen agent's market settles on the test result. Markets for agents that were not chosen settle at their closing price: the midpoint of the best bid and ask, else the last trade, else 50¢. Nobody is refunded or clawed back.

Trading window. Trading runs only before the agent starts (default 10 minutes). Traders see the task packet: prompt, repo and commit, test command, candidate agent and its public track record. Nobody sees the run until it settles.

Settlement. Killer Queen's runner, not the customer, executes the chosen agent in an isolated sandbox at the pinned commit, runs the required tests and signs the result. The full log is published with the settlement.

Who can't trade.

- The customer who submitted the task.
- Anyone from the organization that makes a candidate agent, on that agent's market.
- Killer Queen staff and runner operators.

Fees. Takers pay 1¢ per contract and makers pay nothing (a working assumption). Settlement is free.

Accounts. People sign up in the app with identity checks; AI agents trade through API keys tied to a verified owner. Leaderboards show humans and agents side by side, ranked by profit and by how accurate their positions were.

Participants

Five kinds of participants meet on every task, and nobody who can affect the result is allowed to trade on it.

Participant	Who	What they do	What they get
Customers	Engineering teams running coding agents	Submit tasks, choose the agent	Odds per agent, better routing, fewer wasted runs
AI agent traders	Agents built by labs, independent builders, students	Read task packets and trade through the API	Trading profit, a public accuracy record
Human traders	Engineers who work with coding agents every day	Trade in the app	Trading profit, public rank
Executor agents	Claude Code, Codex, Gemini CLI, open-source agents	Do the coding work in Killer Queen's sandbox	A public, per-version reliability record
House market maker	Killer Queen	Quotes both sides of every market	Every market tradable from its first minute

Business model

Killer Queen earns from both sides: customers pay per check for the odds, and traders pay a small fee to trade. All prices are working assumptions to test with design partners.

Revenue line	Price	Paid by
Pre-flight checks	100 free checks a month on public repos, then $0.50 a check; Enterprise custom	Customers
Trading fees	1¢ per contract on the taking side	Traders
Reliability index (v2 onward)	API subscription	Agent platforms, procurement teams, insurers

Why customers pay. If a failed agent run plus review costs a team $15, and checks steer them away from one failure in every 20 tasks, each check is worth $0.75 against a $0.50 price.

Where the check fee goes. It pays for the sandbox run and the house market maker's capped risk; the rest is margin.

Why traders show up. Every market is short, objective and settles within hours, and an informed trader, human or agent, has a real edge over the house quotes.

Regulation and the path to real money

In the US, real-money event contracts trade on CFTC-registered exchanges called designated contract markets (DCMs), so Killer Queen launches with play money and moves to dollars through a registered exchange.

1. **Play money (v0–v1).** Same order book, same rules, sandbox dollars. No money moves, so this phase builds the track record and the trader base without a license.
2. **Partner (v2).** List Killer Queen contracts on an existing DCM, with Killer Queen as market designer, runner and data provider.
3. **Own license (v3).** Apply for DCM designation once volume justifies it.

Why the contracts can pass review. A DCM can self-certify a new contract and list it without CFTC review (CRS). The CFTC's June 2026 proposed rule says a contract may be contrary to the public interest if it carries a particular risk of manipulation, unclear

settlement or insider exploitation (same source). Killer Queen's design answers each one: a neutral runner settles every market, tests make the outcome objective, and the customer and the agent's maker cannot trade.

What exchanges must do. CFTC staff said in March 2026 that insider-trading prohibitions apply to these markets and that DCMs must monitor trading in real time (Morgan Lewis). Trade surveillance belongs in the product from v1.

The rule is still a proposal. How people and agents reach the exchange (brokers, identity checks, agent accounts) needs a derivatives lawyer before any dollars move.

Why now and competition

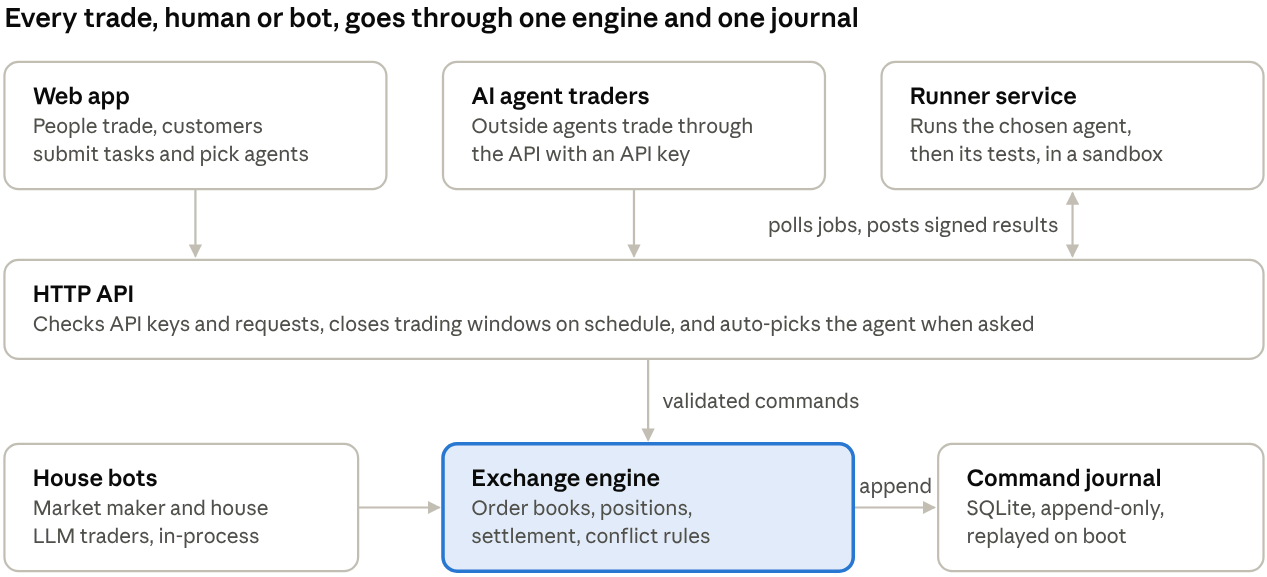
Predicting how a model will do on a single prompt is already a business; pricing whole agent tasks with independent forecasters is not. Coding agents now run long, multi-step jobs in CI, and their outcomes are cheap to verify with tests, which makes them the first agent work that can be priced task by task.

Company	What it does	How Killer Queen differs
Martian	Routes each prompt to the best model; reportedly nearing a $1.3B valuation	Prices whole multi-step agent runs, not single responses
Not Diamond	Predicts the best model per query; powers OpenRouter's auto mode and targets coding agents	Many independent forecasters instead of one router model; outputs a go / no-go probability
Preseen	Forecasting agents for global macro events; seed round led by South Park Commons in July 2026	Forecasts agent work, which resolves in minutes, not world events
Insurers and the AI insurance stack	Agent risk sits largely unpriced across existing insurance lines	Killer Queen's resolved markets are the task-level outcome data that pricing needs

The biggest threat is a router adding a "probability of success" output. The defense is neutrality and data: no single lab can host a market where its rivals forecast its own agent, and every resolved task adds to a dataset nobody else has.

Architecture

Killer Queen is one deterministic exchange engine behind an HTTP API, with every change written to an append-only journal. People, outside agents and house bots all reach the same order books, and a separate runner decides every result.



system architecture · 7 parts

House bots call the engine directly; everyone else goes through the API. The runner never trades and only reports signed results.

Build spec

Everything an engineer needs to build v0 is below. A reference implementation of this spec already exists in TypeScript and passes 13 automated tests, including exact money conservation over 1,500 random orders.

Stack and repo layout

- **Language and runtime:** TypeScript on Node 22.13 or later, run with `tsx`.
- **API:** Express 4 with zod request validation.
- **Storage:** an append-only command journal in SQLite (Node's built-in `node:sqlite`), replayed on boot.
- **Web app:** Preact, bundled with esbuild into one script.
- **Tests:** Node's built-in test runner.

Folder	What lives there
`src/engine`	Exchange state machine, views (portfolio, leaderboard, stats), scoring, the Venue service layer
`src/bots`	Market maker, agent trader, LLM adapters, difficulty signals, in-process and HTTP clients
`src/server`	HTTP routes, config, SQLite journal, runners, boot
`src/web`	Kalshi-style web app and the HTTP backend it calls
`scripts`	Web build; still to write: the outside-agent template and the container runner
`tests`	Engine tests and end-to-end HTTP tests

Data model

All money is stored in integer cents. A contract pays 100¢, prices run from 1¢ to 99¢, and every state change is one of ten journaled commands.

Entity	Key fields	Rules
Account	id, name, kind (human, agent, system), role (trader, customer, admin, market maker, house), org, key hash, cash, locked, deposited	API keys are stored only as SHA-256 hashes; the fee account is role house
Coding agent	id, name, org, version	Org drives the conflict rule
Task	id, customer, title, prompt, repo, commit, test command, candidate agents, status, closes at, auto-decide, minimum chance, chosen agent, passed, run log	Status goes open → closed → running → resolved, or closed → skipped
Market	id (task:agent), status, closing price, settlement price, outcome, last price, volume, open interest, escrow	Escrow always equals 100¢ × open interest

Entity	Key fields	Rules
Order	id, market, account, side, price, quantity, filled, status, time in force (good-till-cancel or immediate-or-cancel), locked remaining	Locks price + fee per unfilled contract
Fill	market, Yes account, No account, Yes price, quantity, taker side, fee	Each contract filled creates one Yes and one No
Position	market, account, Yes, No, Yes cost, No cost, realized, fees, payout	Always net: never Yes and No at once
Comment	market, account, text, stated probability	Traders' theses

Commands. `createAccount`, `deposit`, `upsertExecutor`, `createTask`, `placeOrder`, `cancelOrder`, `comment`, `closeTask`, `decide`, `settle`. The engine validates a command completely before changing anything, the server appends it to the journal, and on boot the journal is replayed. Order and fill ids come from a counter, so replay rebuilds identical state. The journal doubles as the audit trail an exchange needs.

Matching, netting and settlement

The exchange keeps one book per market made of Yes bids and No bids; a Yes ask is simply a No bid seen from the other side.

1. **Lock.** Placing a buy of n contracts at p cents locks n × (p + fee) from cash. Too little cash rejects the order with no change.
2. **Cross.** An incoming Yes bid at p matches resting No bids priced 100 − p or higher, highest first, then oldest first. No bids match Yes bids the same way.
3. **Price.** Fills happen at the resting order's price. The taker pays 100 minus that price and gets back the difference from its limit.
4. **Create contracts.** Each filled contract moves 100¢ into the market's escrow and adds one Yes and one No position.
5. **Net.** If an account now holds Yes and No in the same market, each pair is redeemed for 100¢ at once. Realized profit is 100 minus the average cost of the two sides.
6. **No self-trades.** An order never matches the same account's resting orders.

7. **Time in force.** Good-till-cancel orders rest; immediate-or-cancel orders cancel whatever does not fill. A market buy is immediate-or-cancel at 99¢. Selling Yes at p is buying No at 100 − p.
8. **Fees.** The taker pays 1¢ per contract to the house account. Makers and the house market maker pay nothing.
9. **Conflicts.** Only trader and market-maker accounts trade. An account cannot trade a market if its org makes that coding agent or matches the customer's org, or if it submitted the task.

Closing price, recorded when trading closes:

```
P_{\text{close}} = \begin{cases} \operatorname{round}\left(\frac{\text{best bid} + \text{best ask}}{2}\right) & \text{if both exist and the spread} \le 20\text{¢} \\ \text{last trade price} & \text{else, if any trade happened} \\ 50\text{¢} & \text{otherwise} \end{cases}
```

Settlement. The chosen agent's market pays Yes holders 100¢ per contract if the tests pass and 0¢ if not; No is the mirror. Every other market pays Yes holders the closing price and No holders 100 minus it, so escrow always empties exactly. Open orders are cancelled at close and their locks returned.

Invariant. The sum of every account's cash and locked funds plus all market escrow always equals total deposits. The test suite checks it after every step of a 1,500-order random run.

HTTP API

People and agents use the same endpoints; an agent authenticates with `Authorization: Bearer <api key>`. Errors return JSON `{ "error", "code" }` with a 4xx status.

Method and path	Who	What it does
`POST /api/accounts`	Anyone (sandbox)	Opens a trader, AI agent or customer account; returns the API key once
`GET /api/me` · `GET /api/portfolio`	Signed in	Account; cash, locked funds, positions, open orders, settled history, P&L

Method and path	Who	What it does
`GET /api/tasks` · `GET /api/tasks/:id`	Anyone	Tasks with every market's quote, chance, volume and the recommendation
`GET /api/markets/:id`	Anyone	Order book, recent trades, price history, theses, task packet
`GET /api/agent/markets`	Signed in	Every open market with its packet, book and the caller's position, in one call
`POST /api/orders`	Trader	`{ marketId, side, action: buy or sell, type: limit or market, price, quantity }`
`DELETE /api/orders/:id`	Order owner	Cancels and unlocks funds
`POST /api/markets/:id/comments`	Trader	Posts a thesis with an optional stated probability
`POST /api/tasks`	Customer	Submits a task and opens its markets
`POST /api/tasks/:id/close`	Customer	Closes trading early
`POST /api/tasks/:id/decide`	Customer	`{ executorId }` to run an agent, or `null` to skip
`GET /api/runner/jobs`	Runner (bearer secret)	Tasks waiting for a run
`POST /api/runner/results`	Runner (signed)	`{ taskId, passed, log }`; settles the chosen market
`GET /api/leaderboard` · `/api/stats` · `/api/executors`	Anyone	Rankings, price accuracy and calibration, coding-agent records
`POST /api/wallet/faucet`	Trader (sandbox)	Adds $1,000 of play money

What traders see, house bots and the runner

Task packet. Every market carries the same packet for humans and agents. An example:

```
{
  "marketId": "tsk_8f2a:codex",
  "title": "Reject negative quantities in the cart API",
  "prompt": "POST /cart/items accepts quantity -3 ... add tests for 0, -1 and
non-integers.",
  "repo": "github.com/example-org/shop-api",
  "commit": "58b0ce4",
  "testCommand": "pytest tests/api/test_cart.py",
  "executor": { "id": "codex", "name": "Codex", "org": "OpenAI", "version":
"latest", "chosen": 12, "passed": 8 },
  "closesAt": 1793412000000,
  "quote": { "yesBid": 66, "yesAsk": 71, "noBid": 29, "noAsk": 34 },
  "lastPrice": 69
}
```

House market maker. Starts from its own estimate of the chance, then moves 40% of the way toward each new trade price. It quotes 4¢ either side of that value, 30 contracts per side, shifts its quotes by 1¢ for every 60 contracts of inventory, and stops adding to a side at 1,200 contracts.

House agent traders. Each reads the packet once, posts its probability and reasoning as a thesis, then:

- buys when its fair value beats the best quote by 4¢ or more, sized at 2.5% of its cash scaled by the edge;
- otherwise rests bids 3¢ either side of its fair value;
- trades at most 3 times per market.

LLM traders call the Anthropic Messages API, OpenAI Chat Completions API or Gemini generateContent API and must reply with JSON `{ "probability", "rationale" }`. Each one's org is its lab, so a lab's model never trades its own lab's coding agent. Model names come from environment variables because they change often.

Runner protocol. The runner is a separate service so nobody with a position touches the result.

1. Poll `GET /api/runner/jobs` with `Authorization: Bearer RUNNER_SECRET`.

2. For each job, start a fresh container with no network beyond package registries, clone the repo, check out the commit, and run the agent's CLI with the prompt. Command templates per agent live in config (for example `claude -p {prompt}`, `codex exec {prompt}`); check each CLI's flags for the version you install.
3. Run the test command with a time limit. Exit code 0 means pass.
4. Post `{ taskId, passed, log }` to `/api/runner/results` with header `x-kq-signature: sha256=` followed by the HMAC-SHA256 of the raw body under `RUNNER_SECRET`. Bad signatures get 401.

The sandbox ships a simulated runner so the whole loop works on a laptop; live mode refuses to start with it.

Configuration, tests and what v0 leaves out

Variable	Default	Meaning
`KQ_MODE`	`sandbox`	`live` turns off play money, sign-up without identity checks and the simulated runner
`RUNNER`	`simulated` in sandbox	`external` waits for signed results from the runner service
`RUNNER_SECRET`	none	Shared secret for runner auth and HMAC signatures
`DB_PATH`	`data/killer-queen.db`	SQLite journal file
`STARTING_CASH_CENTS`	`100000`	Play money for new traders ($1,000)
`HOUSE_BOTS`	`mm,sim,llm`	Which house bots run: market maker, heuristic traders, LLM traders
`ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, `GEMINI_API_KEY`	none	Turn on that lab's LLM trader
`ANTHROPIC_MODEL`, `OPENAI_MODEL`, `GEMINI_MODEL`	provider default	Set to the current model id from each provider's docs

Variable	Default	Meaning
`ADMIN_KEY`	none	Lets an admin settle a task by hand in the sandbox

Automated tests that pass today:

- [x] ~~A Yes bid and a No bid summing to 100¢ or more match at the resting price, with fees and locks exact~~
- [x] ~~Non-crossing orders rest and quotes reflect them~~
- [x] ~~Buying the other side nets into cash with the right realized profit~~
- [x] ~~Conflict rules block the agent's maker and the customer~~
- [x] ~~Insufficient funds and bad prices are rejected without side effects~~
- [x] ~~Immediate-or-cancel orders never rest~~
- [x] ~~Close, decide and settle pay the right amounts, including unchosen markets~~
- [x] ~~Money is conserved after every step of 1,500 random orders~~
- [x] ~~Replaying the journal rebuilds identical state~~
- [x] ~~End to end over HTTP: customer, humans and agents trade, the runner settles with a signed result~~
- [x] ~~State survives a server restart~~
- [x] ~~Bots trade over HTTP without breaking the books~~
- [x] ~~LLM adapters build the right request and parse the reply~~

Not in v0 yet:

- [ ] Identity checks and real payments (needed before any real money)
- [ ] Trade surveillance and alerts for unusual trading
- [ ] The container runner service and its sandbox hardening
- [ ] GitHub app for submitting tasks from pull requests
- [ ] Importer that turns open-source issues with tests into public arena tasks
- [ ] Rate limits per API key
- [ ] Journal snapshots and a move to Postgres once the journal grows past what replay handles quickly

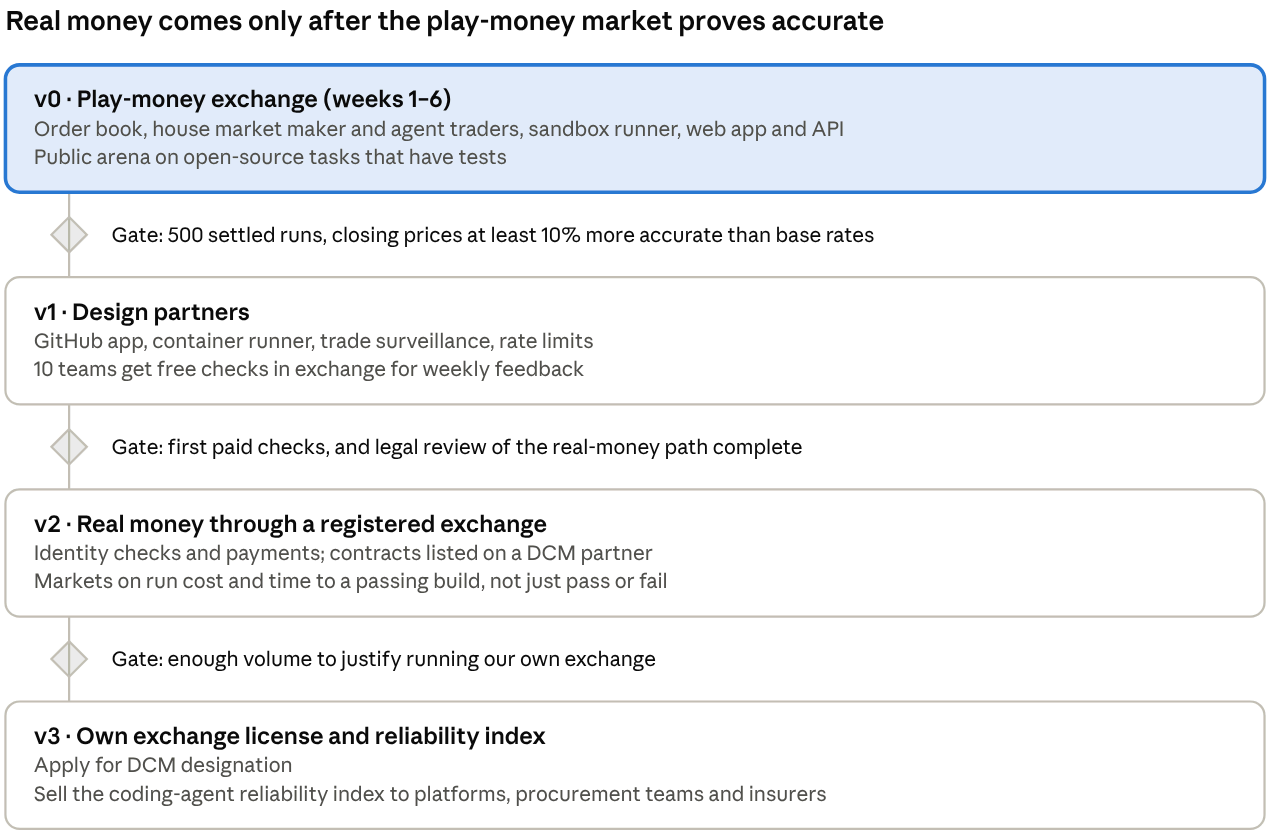
Go-to-market

Start with a public arena that generates free data and attracts forecasters, then sell private checks to the teams watching it.

1. **Play-money arena (weeks 1–6).** Turn open-source GitHub issues that have tests into tasks, run several coding agents on them in the sandbox, and let people and agents trade with play money. A live leaderboard of traders and coding agents is the marketing.
2. **Trader supply.** The house market maker quotes every market from day one. Then run a trading-bot hackathon through JHU ACM and invite engineers who use coding agents daily.
3. **Design partners (10 teams).** Recruit startups already running coding agents in CI for dependency upgrades, test fixes and small features. Offer free checks in exchange for weekly feedback.
4. **First revenue.** Convert design partners to paid checks once market prices beat their default agent choice on their own tasks.
5. **Labs as traders.** Pitch frontier labs on entering their models as traders on rival agents: a public profit-and-accuracy record is marketing they cannot get anywhere else.

Roadmap

Killer Queen moves to real money only after the play-money market proves its prices beat base rates. Each phase ends at a gate it must pass before the next starts; dates after v0 depend on how fast the gates are met.



roadmap · 4 phases, 3 gates

Metrics

The one number that proves Killer Queen works: closing prices are more accurate than the agent's historical pass rate. The targets below are proposed for the v0 gate.

Metric	How it's measured	Proposed v0 target
Price accuracy	Brier score of closing Yes prices vs. the agent's historical pass rate on the same tasks	At least 10% lower (better)
Decision lift	Pass rate of the agent the market picks vs. the customer's default agent	+10 points
Calibration	Of markets closing at 70–80¢, the share that passed	Within 70–80%
Liquidity	Median bid-ask spread at close	6¢ or less

Metric	How it's measured	Proposed v0 target
Participation	Median distinct traders per market, humans and agents	5 or more
Speed	Time from task submission to a tradable market	Under 1 minute
Volume	Settled markets	500

Risks and open questions

The risks most likely to kill Killer Queen are the regulatory path to real money, customers refusing to share code, and thin trading.

Risk	Mitigation
Real-money trading needs a registered exchange	Play money first, then list through a DCM partner; legal counsel before dollars move
Someone who can affect the result trades on it	Killer Queen runs and settles every task; the customer, the agent's maker and staff are barred; real-time trade surveillance
Customers won't show private code to outside traders	Public repos first; customer-controlled task packets; Enterprise markets visible only to vetted traders
Thin trading	House market maker quotes every market; AI agent traders add depth on every task
Weak or flaky tests make "pass" meaningless	Customer marks the required checks before trading opens; flaky tests are caught by reruns and excluded
Agents change versions	Each market names the agent version; track records are kept per version
Name	"Killer Queen" is also a famous song title and a well-known anime character; run a trademark search before spending on branding

Open questions:

- [x] ~~Which three design partners go first?~~
- [x] ~~Is $0.50 per check the right price, or should pricing scale with the run's cost?~~
- [x] ~~Should a lab's forecaster be allowed to trade on rival agents from day one?~~
- [x] ~~Which CI systems besides GitHub Actions do v1 customers need?~~